Multi-Agent Orchestration System (MAOS)

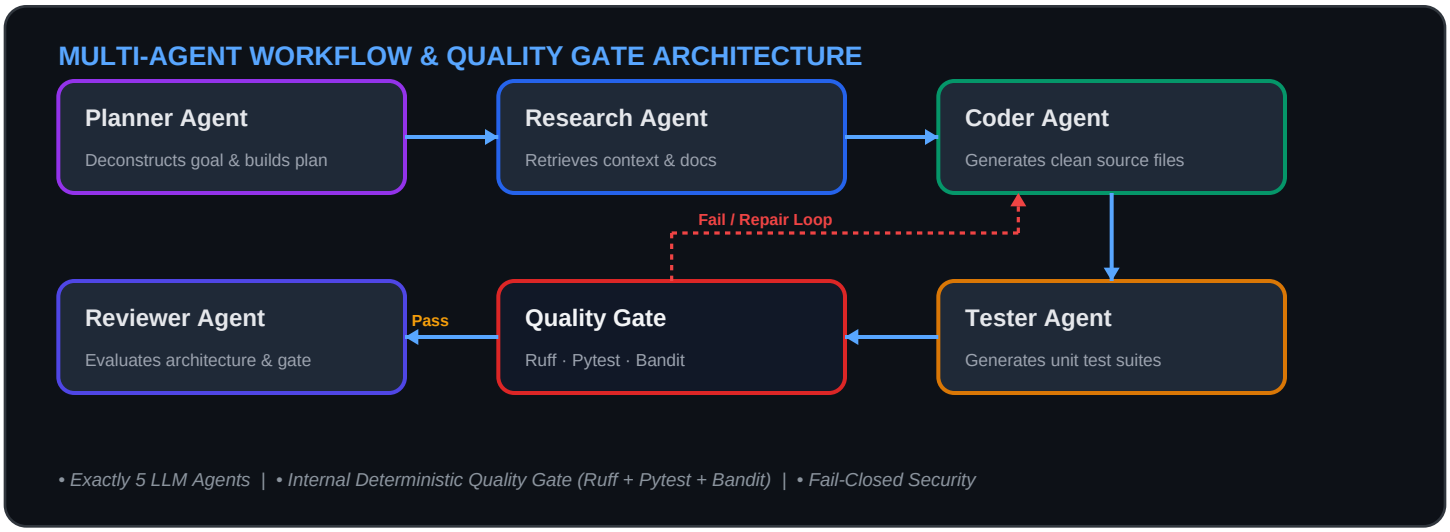
Comprehensive Project Summary & Technical Architecture Specification

1. Executive Overview

The **Multi-Agent Orchestration System (MAOS)** is an enterprise-grade platform designed to autonomously plan, research, write, test, validate, and review complex software systems. Built on an asynchronous micro-orchestration engine with LangGraph, FastAPI, and Next.js, the system coordinates specialized local and cloud-based Large Language Models (Ollama, Groq, OpenAI) to deliver high-quality, production-ready applications with zero hallucinated verification.

2. Core Architecture & 5-Agent Workflow

The system maintains a strict separation of concerns across **exactly 5 LLM agents** supported by an **internal deterministic quality gate**:



Agent / Stage	Role & Responsibilities	Input / Output Artifacts
1. Planner Agent	Deconstructs high-level user requirements into an execution plan, determines required agent stages, and builds a comprehensive file manifest.	**In:** User prompt **Out:** Execution Plan, Manifest, Required Agents
2. Research Agent	Queries local vector embeddings (ChromaDB + FastEmbed) and performs domain knowledge synthesis for optimal architecture patterns.	**In:** Plan + Knowledge Base **Out:** Architectural Research Notes
3. Coder Agent	Generates production-ready code for each file in the manifest with clean artifact separation (no markdown commentary inside code).	**In:** Plan + Research Notes **Out:** Structured Code Artifacts
4. Tester Agent	Designs isolated unit & integration test suites and provides qualitative test coverage analysis without executing subprocesses.	**In:** Generated Code + Plan **Out:** Pytest Test Files + Coverage Analysis
Internal Quality Gate	Deterministic infrastructure executing syntax validation, import resolution, full Ruff linting, Pytest test suites, and Bandit security scans.	**In:** Workspace Sandbox **Out:** Authoritative Pass / Warning / Fail Evidence
5. Reviewer Agent	Performs qualitative review (SOLID principles, maintainability, architectural design) strictly governed by Quality Gate results.	**In:** Code + Gate Evidence **Out:** Final Review & Quality Score

3. Core Subsystems & Technical Capabilities

A. Orchestration Engine & State Graph

Powered by LangGraph, the workflow is structured as a state machine with full pause/resume capabilities, human-in-the-loop approval gates, checkpointing, and dynamic agent routing. If the Quality Gate detects a syntax or test failure, the engine automatically triggers an autonomous **Repair Loop** back to the Coder Agent (up to 3 attempts) before escalating to the Reviewer.

B. Deterministic Quality Gate & Security Scanner

A fail-closed verification pipeline guaranteeing code correctness: • **AST Syntax & Import Validator:** Parses AST trees and validates cross-file module imports against the workspace manifest. • **Ruff Static Analysis:** Executes ruff check --no-cache for linting and undefined symbol detection. • **Pytest Subprocess Sandbox:** Executes tests in an isolated sandbox workspace. • **Bandit Security Engine:** Runs static security scans ignoring legitimate test assertions (-s B101).

C. RAG & Knowledge Vector Store

Integrates **ChromaDB** vector store paired with **FastEmbed** (in-process ONNX embeddings on CPU) and **Nomic Embed Text**. Features recursive text chunking, automatic dimension mismatch auto-healing, and multi-format document ingestion (PDF, DOCX, Markdown, Python, TSX, YAML, SQL).

D. Model Context Protocol (MCP) Server

Implements a standard MCP tool server offering 20+ specialized execution tools across File Systems, PostgreSQL operations, GitHub integrations, HTTP requests, and sandboxed Terminal commands with timeout enforcement and security constraints.

E. Enterprise Security, Multi-Tenancy & AIOps

Built-in JWT authentication, Argon2/bcrypt password hashing, Role-Based Access Control (RBAC: Admin, Lead Developer, Developer, Viewer), Organization/Team tenancy isolation, audit logging, model routing rules, and real-time system health diagnostics.

4. Technology Stack & Infrastructure

Layer	Technologies & Frameworks	Key Role
Backend Core	Python 3.13, FastAPI, Uvicorn, AsyncIO	High-performance asynchronous REST & SSE APIs
AI & Orchestration	LangGraph, LangChain, Ollama, Groq Cloud API	Multi-agent graph scheduling & LLM inference
Database & Vector	PostgreSQL 16, SQLAlchemy 2.0 (asyncpg), Alembic, ChromaDB, FastEmbed	Relational persistence & semantic document retrieval
Frontend & UI	Next.js 16 (Turbopack), React 19, Tailwind CSS v4, Lucide Icons	Real-time interactive dashboard & timeline streaming
Code Verification	Ruff, Pytest, Pytest-AsyncIO, Bandit, AST Inspector	Deterministic static analysis & security scanning